

```

enter any character
end-time (tmr-read timer) )
(println "You reacted in " (tmr-
gettimediff timer start-time
end-time)
" microseconds") ) )

```

Testing the examples

We have stored the above two source codes (led.l and timer1.l) in a memory card as explained in this section. To test these two examples, you need Hempl software along with gcc, SCons, dfu-programmer and minicom software running on your Linux system. For this, follow the steps given below:

Installing Hempl software. Get the latest Hempl distribution from github. Make a working directory, say, hacking, for this on your Linux machine (Ubuntu version 13.04)

```

$ cd hacking
$ git clone          www.github.com/
                    simplemachines-italy/hempl.git
$ cd hempl

```

Now, you should be in Hempl directory.

Installing gcc for AVR32. We require a gcc for AVR32 device. Get that from www.en.wikibooks.org/wiki/Hempl/Compiling_Hempl. Select Install an AVR32 cross-compiler/Fetching and unpacking the toolchain. Follow the instructions and get the toolchain installed.

Place the toolchain in home directory. It could be something like:

```
/home/avr32-gnu-toolchain-linux_x86/
```

From Hempl directory, make sure the toolchain is in the environment path. Now, do a simple export PATH.

```
$export PATH=$PATH:HOME/raman/avr32-
gnu-toolchain-linux_x86/bin
```

And to check if gcc is running:

```
$avr32-gcc --version
```

It will show the version of gcc.

Installing SCons. Install SCons using the following command:

```
$ sudo apt-get install scons
```

Compiling and generating hex codes. From Hempl directory, do the following to compile the code base. For compiling AT32UC3A0256 de-

vice, enter the following command:

```
$ scons cpu=at32uc3a0256 board=mizar32
toolchain=avr32-gcc prog
```

This step will produce the machine code for running PicoLisp on Mizar32 board. The compilation will take a few minutes. When done, a hex file called hempl_at32uc3a0256.hex will be generated. This hex file will be burnt into the MCU on Mizar32 board. To understand how to flash the program, refer www.en.wikibooks.org/wiki/Hempl/Flashing_firmware

Installing dfu-programmer software. Use a program called dfu-programmer. The MCU on Mizar32 board comes with dfu bootloader. It occupies 8kB on the MCU's flash memory. Please note that a switch (SW2) is required to put the chip in program mode. As soon as we give the following command, program starts running.

```
$ sudo dfu-programmer at32uc3a0256 start
```

Note. Before programming the device, power on Mizar32 board (or press reset button SW1) while holding the user button (SW2) depressed. For details, refer Mizar32 quick-start guide available on the Internet.

Installing minicom. To work with this from the shell, install a terminal emulator called minicom. It works like a charm on GNU/Linux system. To install this, give the command:

```
$ sudo apt-get install minicom
```

There is a need to configure the minicom session with the following: baud rate: 115200, eight data bits, parity none, one stop bit using following command:

```
$ sudo minicom -s
```

Now we are able to talk to our MCU via minicom. Search for the device in /dev. In our case, the device is in /dev/ttyACM0. Confirm the registration by doing a dmesg on your GNU/Linux. Enter following command:

```
$ sudo minicom -D /dev/ttyACM0
```

With this, we are actually communicating with our MCU in serial over USB mode.

EFY Note

The source code of this project is included in this month's EFY DVD and is also available for free download at source.efymag.com

Enter few commands like ls or cat and see your shell.

```
Hempl#
```

You can fire PicoLisp session like this:

```
Hempl# picolisp
```

You can write PicoLisp interactively.

The other option is to invoke PicoLisp with a file argument stored in the micro-SD memory card. In this example, we have stored the source codes in the memory card in .l formats (led.l and timer1.l). The SD card is then inserted into the slot provided on Mizar32 board.

To run toggle_LED code led.l, give the following command:

```
Hempl# picolisp /mmc/led.l
```

To see output, press the user button (SW2). The blue LED light on the board will glow and will go off when you release the button. Next, open the minicom and run the reaction timer code (timer1.l) and give the command:

```
Hempl# picolisp /mmc/timer1.l
```

After getting the : symbol, type the following:

```
:(reaction-timer)
```

Please note the parenthesis. It is the syntax used to invoke Lisp function defined in timer1.l.

As soon as you run timer1.l, press any key on the keyboard. Time displayed on the terminal is the recorded time for showing how fast your hand is. You can press the key as many times as you would like and the corresponding time will be displayed on the terminal. **EFY**



Raman Gopalan is a volunteer programmer at SimpleMachines, Italy, and is currently hacking on PicoLisp for the Mizar32 computer